

A field report on dragging an AI-generated app to ship-ready

The Goose Rescue

human.

Goose arrived as a WHOOP-style recovery tracker: a fat Rust core doing the math, a thin SwiftUI skin on top, and a hand-rolled JSON bridge stitching the two together. Ten commits across two days. One of them literally read *"Merge branch 'codex-server'."* Nobody had ever run it.

The architecture was genuinely good — separation of concerns, exhaustive tests. But it carried the classic tells of machine-written code: an FFI boundary that turned any Rust hiccup into an instant app crash, 120 lint warnings nobody had cleaned, 8,000-line files, no Apple privacy manifest, and test fixtures that were never committed. Clever, untested, and one panic away from a black screen.

01

Don't let the core take the whole app down.

02

bridge.rs — FFI boundary

```
// BEFORE — a panic in Rust = SIGABRT = the app dies on screen
panic = "abort"          # Cargo.toml

// AFTER — panics are caught and returned as structured errors
panic = "unwind"         # Cargo.toml

#[no_mangle]
```

Both `extern "C"` entry points now sit behind `catch_unwind`. A crash in the math layer becomes a JSON error the Swift side can show — not a dead app. This single change converts the app's most fragile seam into its most defensive one.

120 →

03

0

Clippy warnings on the library, taken to zero — real fixes, not blanket suppressions.

`is_some() + unwrap() → if let Some(..)`

`let-else → the ? operator (×4)`

`manual loop → while let`

`.max(0.).min(180.) → .clamp(0., 180.)`

`collapsed duplicate else-if blocks`

one `.expect()` **kept** — bounds invariant proven, FFI now guards it

Four reasons Apple says no. Now zero.

04

✕→✓ Missing privacy manifest

→ PrivacyInfo.xcprivacy (UserDefaults CA92.1, FileTimestamp C617.1)

✕→✓ No display name

→ CFBundleDisplayName "Goose"

✕→✓ False "Amplitude SDK" match

→ renamed a waveform var; verified no tracking SDK exists

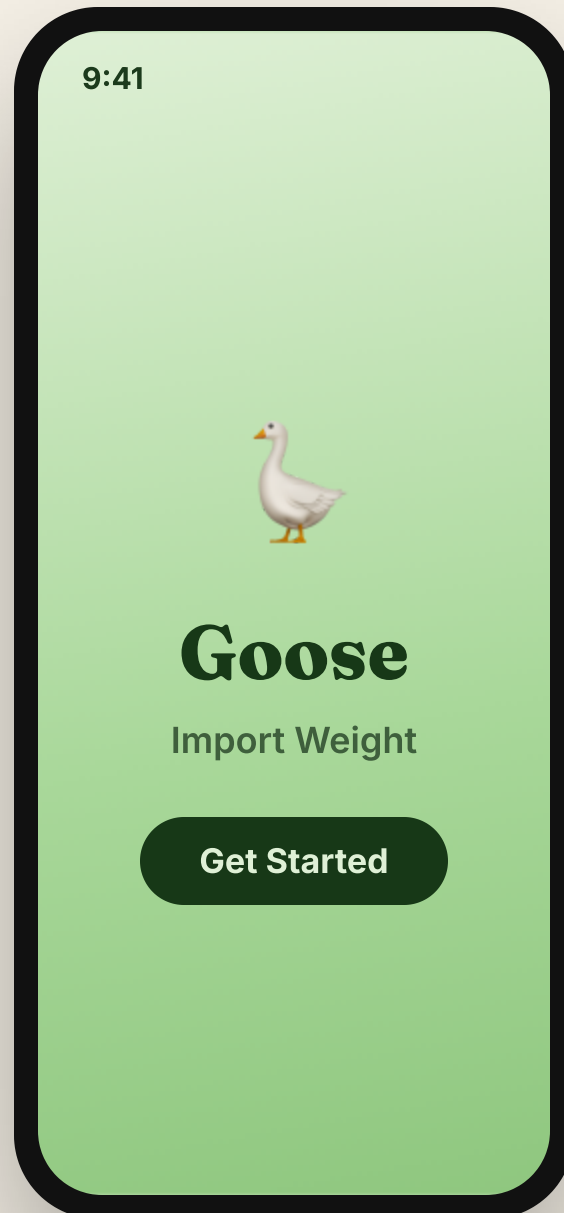
✕→✓ Manifest not in target

→ wired in via pbxproj, confirmed bundled in the .app

GREENLIT — 0 CRITICAL

Downloaded the 8.52 GB iOS 26.5 simulator runtime, rebuilt, and watched it boot.

05



WHAT WE DID NOT PRETEND WAS DONE

Honest about the gap.

STILL OPEN

Shipping-ready is not the same as finished. The reference frame here is the full spec, not the distance travelled — so the open items stay on the board, named:

16 Swift concurrency / data-race warnings — 15 clustered in `GooseAppModel+NotificationPipeline.swift`, 1 in `HealthDataStore.swift`. The real remaining bug surface.

45 test-code clippy warnings + uncommitted test fixtures — the full suite can't run until those are generated.

Nothing pushed. No branch, no PR — by rule, not by accident. Awaiting approval.

A green build that crashes on launch is not done. This one launched clean — but the races are real and unfixed, and we said so.

Source · `swiftc build warnings + cargo clippy --all-targets` · the “compared-to-what” rule



Diagnosed, hardened, run.

An AI wrote Goose in a weekend. It took a human-directed pass — panic-safety at the seam, a clean lint ledger, a real privacy manifest, and an actual launch on a real simulator runtime — to make it something you could put in front of Apple. The wins are real. So is the list of what's left.

THE GOOSE RESCUE · ENGINEERING EDITION № 001 · JUNE 3, 2026 · RENDERED BY /MAGAZINE